pif_eth — Project Report

8b/10b line-coded Ethernet transmitter over the PRU Peripheral Interface, generated on 2026-07-20 in the `pru_simulator` project.

---

1. Initial prompt

The project was requested with the following prompt (verbatim):

> generate a project folder under source folder called pif_eth. This folder hosts a project which transfers Ethernet frame which is 8/10B line coded over 3 peripheral interface using 125 Mbit line rate - 100 Mbit raw Ethernet. The 8B/10B line code is a single LUT using LBCO from ICSS DRAM0. Use PRU0 single core implementation. Example Ethernet frame has two options. Option 1 using pseudo random number generate from Byte 1 to byte 128 + 4 bytes of CRC32. The frame is meant for bit error rate testing and has no pre-amble and no Ethernet header. All payload bytes are random number. Second frame is a standard UDP packet with Ethernet header UDP header, UDP payload "Hello World Text" and CRC32 which is Ethernet frame check sum. The CRC32 in a second stage will use broadside widget CRC32. In first implementation can be firmware generated. Data flow option 1: pseudo random number generator with defined seed -> pack into 128 byte Ethernet Frame. Calculate CRC32 in firmware which is bytes 125-128. The 128 bytes reside in memory. Send of 8/10B line coded frame is in realtime. Load 32 bit from memory, LUT to get 40 bit, send 32 bits to TX fifo of peripheral interface. Wait for Fifo half empty, load next bytes from DRAM, LUT and compile next 16 bits which is 8 bits remaining from previous and 8 bit from new LUT, continue like that till end of frame. Test the code with MCP server for PRU. Ideally generate a wireshark trace for the frames generated. Keep a frame counter and run the test like 100 frames.

Four points in the prompt were ambiguous and were clarified before implementation (see §2).

---

2. Design choices and decisions

2.1 Clarified with the requester

Question	Decision	Rationale
"over 3 peripheral interface" — one channel or striped across three?	**Single channel (ch0)**, one continuous bit stream	Matches the byte-repacking data flow in the prompt; striping deferred
"single LUT" vs. true 8b/10b (disparity-dependent, two codes per byte)?	**True 8b/10b with running disparity**, single LUT holding both RD variants	Requester wanted realistic line code; still a single DRAM0 table
BERT frame length — 128 or 132 octets?	**132 octets** (128 payload + 4 CRC)	Requester's choice
Wireshark trace scope	**Both** frame types to pcap	BERT is malformed Ethernet (expected); UDP dissects cleanly

2.2 Line code

- IBM/ANSI **8b/10b** (Widmer & Franaszek): 5b/6b on the low 5 bits + 3b/4b on the high 3 bits, transmitted `a b c d e i f g h j` (a first on the wire), with the D.x.A7 alternate to break runs of five.
- The reference encoder (`codec.py`) is validated by **invariants** rather than by transcribing the ISO tables: running disparity stays ±1 at every symbol boundary, per-symbol disparity is −2/0/+2, no run exceeds 5 bits, and decode is a bijection over all 256 octets in both disparity contexts. The firmware and the host decoder only need to agree with this reference (hardware interop is out of scope for a simulator BER test).
- **K28.5** commas are emitted as idle fill / inter-frame delimiters, which also give the decoder symbol alignment.

2.3 Line rate

The bit clock is `source / ((frac+1)·(div+1))`; exact 125 MHz is not integer-divisible from the 200 MHz core clock. The model uses `TXCFG` div = 7 → **25 MHz**, which is representable and leaves the single core ample headroom to keep the 4-deep TX FIFO fed. Because the encoder and the host decoder share the configured period, the absolute rate does **not** affect the byte stream or the measured BER; **125 Mbaud** remains the documented physical target.

A 2026-07-21 follow-up demonstrates the 125 Mbaud target is reachable at a 250 MHz core clock (divider n=2) with a rewritten TX loop — see **§8**.

2.4 Frame structure

- **BERT (option 1)** — no preamble, no header. 128 octets from a firmware **xorshift32** PRNG (defined seed, state persists across frames) + 4-octet **CRC-32** = 132 octets.
- **UDP (option 2)** — Ethernet/IPv4/UDP, payload `"Hello World Text"`, padded to the 60-octet Ethernet minimum + 4-octet FCS = 64 octets.
- **CRC-32** was firmware-generated in the first stage (bit-serial, reflected `0xEDB88320`, init/final `0xFFFFFFFF`), identical to `zlib.crc32`. The second stage the prompt asked for is now done: the FCS runs on the ICSSG CRC16/32 broadside accelerator (2026-09-24, **§9**).

2.5 Firmware architecture

- Single-core **PRU0**, channel 0, **continuous** TX mode.
- **Self-configuring**: writes GPCFG0/TXCFG/CH0CFG0 itself; no host register setup. Channel 0 is selected with a byte-2 R30 write — a byte-0 write would strobe the TX FIFO and inject a stray 0x00 byte (this bug was found and fixed during bring-up, see §5).
- Each Ethernet frame is a **separate continuous-mode burst**: payload/CRC are generated *before* `tx_go`, so the FIFO never under-runs during generation (an unbroken multi-frame stream would starve while the PRNG/CRC loops run).
- Subroutines with **fixed return-address registers per nesting level** (`r29 → r28 → r26 → r27`) stand in for a call stack the ISA lacks.
- A per-frame **go-flag handshake** (`0x0418`) lets the host release and capture one frame at a time.

2.6 Host verification / capture

The perif TX timeline advances as PRU0 steps, recording timestamped serial-line transitions. The driver releases one frame, steps until the frame counter increments, then reconstructs that burst's bits by sampling the recorded line at bit centres (`go + (k+½)·period`), 8b/10b-decodes them, checks them against the golden reference (**BER =**

0), and writes the decoded frames (FCS stripped) to a classic-format pcap (Ethernet link type). The firmware ends in a spin (not `halt`) so the perif keeps advancing while the last burst drains.

---

3. Files generated

All under `source/pif_eth/` unless noted (≈1,400 lines total).

File	Lines	Purpose
`pif_eth_tx.asm`	260	PRU0 firmware: self-config, PRNG, CRC-32, 8b/10b encode, stream
`codec.py`	155	8b/10b encode/decode + DRAM0 LUT builder (golden reference)
`crc32.py`	37	Ethernet FCS (bit-serial + `zlib` references)
`pif_eth_crc32.inc`	50	shared `crc32_core`, bit-serial software version (now the fallback, §9)
`pif_eth_crc32_hw.inc`	63	2026-09-24: shared `crc32_core` on the CRC16/32 broadside accelerator, included by all firmware (§9)
`prng.py`	30	xorshift32 PRNG reference
`frames.py`	83	BERT and UDP frame builders
`decoder.py`	63	bit/symbol stream → frames (comma-delimited)
`pcap.py`	30	minimal classic-pcap writer (Ethernet link type)
`driver.py`	191	run firmware on the sim, decode, check BER, write pcap
`seed_ui.py`	74	seed the running UI simulator's DRAM0 over WebSocket
`__init__.py`	5	package docstring
`README.md`	96	package overview / quickstart
`PROJECT_REPORT.md`	—	this document
`tests/test_pif_eth.py`	222	18 tests (pure-Python + firmware + MCP)
`docs/superpowers/specs/2026-07-20-pif-eth-8b10b-tx-design.md`	94	design spec
`docs/superpowers/plans/2026-07-20-pif-eth-8b10b-tx.md`	56	implementation plan / as-built
`pif_eth_tx_n2.asm`	—	2026-07-21 follow-up: batched/unrolled TX loop, all FIFO checks intact (§8)
`pif_eth_tx_n2_skipchecks.asm`	—	2026-07-21 follow-up: as above, 2 FIFO checks removed to hit n=2 (§8)
`docs/superpowers/specs/2026-07-21-pif-eth-n2-125mbaud-design.md`	—	design note for the n=2 follow-up

Also modified: `.gitignore` (ignores the generated `source/pif_eth/traces/` pcap artifacts).

DRAM0 memory map

Address	Contents
`0x0000`	8b/10b encode LUT, 256 × u32
`0x0400`	`num_frames` (u32)
`0x0404`	`mode` (u32): 0 = PRNG payload, 1 = preloaded payload
`0x0408`	`prng_state` / seed (u32)
`0x040C`	`payload_len` (u32)
`0x0410`	`frame_counter` (u32, published)
`0x0414`	`burst_pushed` bytes (u32, published)
`0x0418`	go flag (u32, host→firmware handshake)
`0x0500`	frame core buffer (payload + 4-byte FCS)

---

4. Testing

The strategy is **golden-reference first, then firmware**: a pure-Python reference layer defines the exact byte-level behaviour the firmware must reproduce, so firmware bugs surface as byte mismatches against a known-good model. `tests/test_pif_eth.py` has **18 tests** (all green; full repo suite 1159 passed):

Pure-Python reference (no simulator)

- `test_encode_decode_roundtrip_all_bytes_both_rd` — bijection over all 256 octets, both disparities.
- `test_running_disparity_stays_bounded` / `test_no_run_longer_than_five` — 8b/10b legality invariants.
- `test_comma_flips_disparity`, `test_dram0_lut_matches_encoder`, `test_stream_decode_with_commas`.
- `test_crc32_bitwise_matches_zlib`, `test_crc32_check_value` (`"123456789"` → `0xCBF43926`), `test_prng_*`, `test_bert_frame_shape`, `test_udp_frame_valid_length_and_fcs`, `test_full_frame_line_roundtrip`.

Firmware on the simulator (single-core PRU0)

- `test_firmware_self_configures_perif_ch0` — GPCFG/TXCFG/CH0CFG0 written by firmware; continuous mode.
- `test_firmware_prng_and_crc_in_dram` — reads DRAM0 and checks the firmware's PRNG payload and CRC-32 against the reference (isolates codegen from line coding).
- `test_firmware_bert_roundtrip_zero_ber` / `test_firmware_udp_roundtrip_and_valid_ethernet` — end-to-end BER = 0; the UDP capture is checked for a valid IPv4 ethertype and the `"Hello World Text"` payload.
- `test_pcap_output_roundtrips` — pcap header + record count.
- `test_via_mcp_server` — drives the firmware **through the MCP server wrapper** (`PRUSimulatorMCP.pru_load` / `pru_step` / `pru_memory`) and verifies the frame the firmware built in DRAM0. This satisfies the "test with MCP server" requirement.

Full run and Wireshark trace — `python3 source/pif_eth/driver.py 100` transmits 100 frames of each type (BER = 0 for both) and writes `traces/pif_eth_{bert,udp}.pcap`. Verified externally with `tshark`:

```
$ tshark -r source/pif_eth/traces/pif_eth_udp.pcap -c 1
    1   0.000000  192.168.0.1 → 192.168.0.2  UDP 60 1234 → 5678 Len=16
```

with the UDP payload dissecting as `Hello World Text.`

Run the suite:

```
python3 -m pytest tests/test_pif_eth.py -q      # 18 tests, ~0.8 s
```

---

5. Notable issues resolved during bring-up

- **8-bit AND immediate** — the 10-bit `0x3FF` symbol mask cannot be an immediate; it lives in a register.
- **Stray FIFO push** — `ldi r30, 0` strobes byte 0 of R30, which the perif treats as a TX FIFO push of `0x00`, corrupting the *first* burst only (the second frame was already perfect). Fixed by selecting ch0 with `ldi r30.b2, 0`. This was found by dumping the first burst's sampled bits and seeing exactly 8 leading zero bits before the expected comma.
- **Continuous-mode under-run** — generating payload/CRC mid-stream starves the serialiser; resolved by making each frame its own burst.
- **Timeline advance** — the perif only advances while PRU0 steps, so the firmware spins (not `halt`) and `drain_wait` keeps stepping until the burst fully serialises before the frame counter is published.

---

6. Token / effort usage

Exact token telemetry is not exposed to the agent, so the figures below are an **approximate** account of where effort went, by phase. The dominant costs were (a) reading the existing simulator/perif source to learn the conventions and (b) the iterative firmware bring-up loop.

Phase	Activity	Relative effort
Exploration	Read perif model (`peripheral_interface.py`, `perif_channel.py` ~477 lines, `perif_registers.py`), MCP server, ISA handlers, existing perif firmware/tests, memory + constant-table config	~30%
Design / clarification	4 clarifying questions; reconciling data flow with the model (single-burst vs. per-frame, timeline advance)	~10%
Python reference layer	codec/crc/prng/frames/pcap/decoder + first test run	~15%
Firmware	~260-line assembly + parse/smoke iterations	~15%
Bring-up / debugging	2 end-to-end debug cycles (stray-push + alignment), 100-frame run	~15%
Docs / tests / report	spec, plan, README, this report, UI helper	~15%

Order-of-magnitude, the session generated ~1,400 lines of code/docs and read several thousand lines of existing source; total token consumption was on the order of a few hundred thousand tokens, weighted toward source reading and the edit/test iterations. The generated code was kept small and reference-driven specifically to keep the

debug loop (and thus token spend) short — the firmware was validated against the Python golden model rather than by trial and error.

---

7. Running with the UI simulator

The UI (`ui/server.py`) keeps a single shared `Simulator`, so the browser and a small seeding client operate on the same DRAM0. The firmware needs the 8b/10b LUT and a control block in DRAM0, which the seeding helper writes over the UI's `write_memory` WebSocket action.

Steps

1. **Start the dashboard** (terminal 1):

   ```
   python ui/server.py
   ```

   Open http://localhost:8080.

2. **Load the firmware** in the browser: select core **PRU0**, paste the contents of `source/pif_eth/pif_eth_tx.asm` into the editor, click **Load**. (`Load` parses the program but does not clear DRAM0, so seed order is flexible — but re-seed after any **Reset**, which clears memory.)

3. **Seed DRAM0 and arm one frame** (terminal 2):

   ```
   python3 source/pif_eth/seed_ui.py bert     # or: udp
   ```

   This writes the LUT (`0x0000`), the control block (`0x0400…`) and sets the go flag (`0x0418 = 1`), arming exactly one frame.

4. **Run.** Click **Run** (there is no step-budget field — Run is a Run/Stop toggle that streams batches of 1,000 instructions on a timer until you click **Stop** or the core halts). A BERT frame is ≈ 20,000 instructions, so it completes in a fraction of a second; wait a moment, then click **Stop** (the firmware then spins waiting for the next go flag). The I/O window switches to the **Peripheral Interface** view as the firmware writes GPCFG; you can watch channel 0's TX FIFO activity. (For a precise count instead, send a WebSocket `{"action":"run","core":"pru0","max_steps":30000}`.)

5. **Observe results** in a **Memory** panel:

   - `0x0410` — frame counter (increments to 1),
   - `0x0414` — bytes pushed for the burst,
   - `0x0500` — the 132-byte (BERT) / 64-byte (UDP) frame the firmware built.

6. **Next frame:** re-run `seed_ui.py` (it re-arms the go flag) and click **Run** again; the counter advances. Repeat as desired.

Headless full run (recommended for the 100-frame BER test + traces):

```
python3 source/pif_eth/driver.py 100
tshark -r source/pif_eth/traces/pif_eth_udp.pcap
```

> Note on the UI path: the browser's own **Run** button drives a single core and is ideal for stepping and inspecting one frame at a time. The full 100-frame BER measurement and pcap generation are done by `driver.py`, which also performs the per-frame capture/decode the browser UI does not.

---

8. Follow-up: 125 Mbaud (n=2) experimental firmware (2026-07-21)

Status: experimental — not integrated into `pif_eth_tx.asm`, `driver.py`, or the test suite; standalone reference artifacts.

8.1 Goal

§2.3 above documents the production firmware at 25 MHz (TXCFG div=7 @ 200 MHz core) because exact 125 MHz isn't integer-divisible from a 200 MHz clock. A separate piece of work (the PRU core speed selector, `docs/superpowers/specs/ 2026-07-20-pru-core-speed-selector-design.md`) added selectable core clocks up to 333 MHz, which makes the architecture's actual 125 Mbaud target reachable: `n = (frac+1)*(div_factor+1)`, `bit_clock = core_clock/n`, so divider **n=2 at 250 MHz core** gives exactly 125 MHz. This follow-up tests whether the existing firmware design can sustain that rate.

8.2 Firmware variants

- **`pif_eth_tx_n2.asm`** — `pif_eth_tx.asm` with TXCFG set to div=1 (n=2) and `send_frame`'s bit-packing rewritten: 4-octet batched `LBBO` loads (frame length is a multiple of 4 for both BERT=132 and UDP=68) and fixed-shift unrolled packing — the bit-remainder cycles 0→2→4→6→0 deterministically every 4 pushes, so each push's shift amount is a compile-time constant rather than a runtime `nbits`-tracked value. Applied to **both** the data loop and the leading/trailing K28.5 commas (the commas, still using the old `push_symbol` in an early iteration, were the first thing to underrun). All 5 FIFO-full checks intact.
- **`pif_eth_tx_n2_skipchecks.asm`** — identical, except the FIFO-full check is removed on the data loop's phase 0 and phase 1 (the two single-byte-emit phases); phase 2 and phase 3 keep their checks.

8.3 Results

Measured in-simulator (`Simulator(config_path=...)` with `pru_clock_mhz=250`) via the same `codec`/`decoder` golden-reference path used in §4:

Firmware	n	Bit rate	Result
`pif_eth_tx_n2.asm` (all checks)	2	125.00 MHz	**FAIL** — underrun, then deadlock (the FSM goes idle after a premature `_finish_frame()`; later pushes spin forever waiting for a drain that never resumes)
`pif_eth_tx_n2.asm` (all checks)	3–8	83.33–31.25 MHz	**PASS**, BER=0 at every step
`pif_eth_tx_n2_skipchecks.asm`	2	125.00 MHz	**PASS**, BER=0 over 100 BERT + 100 UDP frames
`pif_eth_tx_n2_skipchecks.asm`	3	83.33 MHz	**FAIL** — BER≈0.68, `tx_overrun=True`
`pif_eth_tx_n2_skipchecks.asm`	4	62.50 MHz	**FAIL** — BER≈0.68, `tx_overrun=True`

Silicon (AM243x LP, 250 MHz, 2026-10-10): n=4/6/8 with the all-checks firmware and n=2 with `skipchecks` all pass 100/100 BERT frames, but n=2 only with a trailing zero byte after `flush_pad`. Without it the perif TX serializer drops the last pushed byte at n=2: the final comma loses its last 4 bits and about half the frames show `symbol_errors=1` (data, CRC and BER clean; `tx_pushed=173` confirmed PRU0 pushed everything). The simulator does not model this. The pad byte is not counted in `tx_pushed`.

8.4 Root cause: the n=2 underrun, and why removing 2 checks fixes it

At n=2 the perif channel drains the TX FIFO deterministically every `8*n = 16` core cycles/byte (exact integer, no jitter — same-clock sampling, `perif/perif_channel.py`). The fully-checked, fully-unrolled loop's production cost sits right at that boundary (hand-analysis: ~15.6 avg / 16.0 worst-case cycles/byte). Single-stepping the simulator shows the actual failure is a **2-cycle race**: the FIFO-full check passes (not full), but the drain's next bit-edge — landing deterministically in the gap between the check and the following `mov r30.b0,r5` — pops the FIFO before the byte lands, finds it empty, and ends the burst early. Removing the check on phases 0 and 1 saves 2 cycles per push at exactly the two points that mattered, closing that gap. This was confirmed by single-stepping the simulator, not by the hand cycle-tally alone.

8.5 Critical caveat — a single-divider hack, not a general speedup

The FIFO-full check is not just a safety margin — it is the mechanism that makes the firmware's push rate automatically track whatever the configured drain rate is. Removing it fixes the firmware at a constant production rate tuned to *just barely* match n=2. The moment the drain is slower (n=3, n=4 — both tested), nothing throttles the firmware anymore, and it pushes into an already-full FIFO every few bytes: `tx_overrun=True`, ~68% BER, frames silently truncated to a fraction of their expected length.

`pif_eth_tx_n2_skipchecks.asm` must only ever be run with `pru_clock_mhz=250` and TXCFG div_factor=1 (n=2). If `memory.cfg`'s `pru_clock_mhz` or the firmware's TXCFG divider ever changes, this firmware will not error or hang loudly — it will **silently produce wrong data** (overrun, not a crash) unless re-validated at the new clock/divider combination first.

8.6 Recommendation

- For a **robust** speedup that stays correct across clock/divider changes, use `pif_eth_tx_n2.asm` (all checks) at n≥3 — a genuine ~4x improvement over the original 25 MHz design (83.33 MHz @ n=3), with the same self-adapting safety margin the original design relies on.
- `pif_eth_tx_n2_skipchecks.asm` is kept as a validated, working example of hitting the full 125 Mbaud architectural target — but it is hard-pinned to n=2 and should be treated as a one-off, not a drop-in replacement for `pif_eth_tx.asm`.
- Neither file is wired into `driver.py` or `tests/test_pif_eth.py`; both remain standalone artifacts under `source/pif_eth/` for reference/future work. A genuinely robust n=2 firmware would need to cut the bit-packing loop's cycle cost further, or use a hardware-level multi-byte FIFO push — which does not currently exist in the perif model or the documented `references/endat/ENDAT_INTERFACE_SPEC.md`.

Full design note: `docs/superpowers/specs/2026-07-21-pif-eth-n2-125mbaud-design.md`. Cross-PC handoff: `docs/handoff/2026-07-21-speed-selector-and-pif-eth-n2.md`.

---

9. Follow-up: CRC-32 on the broadside accelerator (2026-09-24)

9.1 Goal

The original prompt made a broadside CRC32 the second stage ("The CRC32 in a second stage will use broadside widget CRC32"). The simulator's `dev` branch now models the ICSSG CRC16/32 accelerator (`xfr/crc_accelerator.py`, XFR device 1; AM64x/AM243x TRM SPRUIM2H §6.4.6.2.2.1, Table 6-429; `references/icss_m_xfr_crc_functional_spec.md`). This follow-up moves every pif_eth FCS computation onto that accelerator.

9.2 Implementation

- **`pif_eth_crc32_hw.inc`** is a new `crc32_core` with the same contract as the software routine: entry `r21` = buffer, `r8` = length; exit `r20` = FCS, `r21` = end of buffer; return via `r26`. It works as follows:
    1. `XOUT 1, &r25, 1` with `CRC_CFG = 0x01` selects CRC-32. That write also seeds `0xFFFFFFFF`.
    2. A zero-overhead `LOOP` runs `LBBO &r29` / `XOUT 1, &r29, 4` for each 32-bit word.
    3. Two `NOP`s (TRM rule), then `XIN 1, &r28, 4` (`CRC_DATA_32_BFLIP`). Silicon showed the `R29` accumulator is not reflected; its 32-bit mirror is the reflected Ethernet CRC. The engine applies no final XOR, so the firmware does `NOT`.
- **Uneven lengths.** The TRM says one CRC session must keep one write width. So 1–3 trailing bytes run as a second, byte-wide session seeded via `CRC_SEED` (`R28`) with the word session's raw `R29` result. BERT (128) and UDP (60) never need this; RX with `payload_len = 201` does.
- **Register window.** The broadside window is fixed at `R25`–`R29`, which overlaps the firmware's return-address registers `r28`/`r29`. In RX both are live during `rx_crc_check`, so `crc32_core` saves them in `r22`/`r24` and restores them.
- **Firmware changes.** `pif_eth_tx.asm` and `pif_eth_rx_o1_raw.asm` only swap their `.include`. `pif_eth_tx_n2.asm` and `pif_eth_tx_n2_skipchecks.asm` each had an inlined copy of the bit-serial loop. Both now call the shared routine, so all four images use one CRC implementation. `pif_eth_crc32.inc` stays as the software fallback for a core without the accelerator.

9.3 Cycle savings (measured on the simulator)

Cycles are counted from the `jal` into `crc32_compute` / `rx_crc_check` to its return, by single-stepping the simulator. They include DRAM stall cycles (`read_latency = 2` in both configs used).

Routine	Payload	Software (bit-serial)	Accelerator	Saving
TX `crc32_compute`	128 B (BERT)	9 488 – 9 552	**181**	~52× (−9.3k cycles)
TX `crc32_compute`	60 B (UDP)	4 456	**96**	~46×
RX `rx_crc_check`	128 B (BERT)	9 526 – 9 558	**187**	~51×
TX / RX	201 B (byte tail)	14 864 / 14 873	**283 / 289**	~52×

- **Per octet.** The software loop costs about 74 cycles per octet. The count depends on the data, which is why it varies between frames. The accelerator costs 5 cycles per 4-octet word (`LBBO` 1 + 2 DRAM stall, `XOUT` 1, `ADD` 1; `LOOP` adds nothing), about 1.25 cycles/octet. The fixed setup and readout add about 20 cycles. The cost is now deterministic. The remaining loop is bound by DRAM read latency, not by the CRC.
- **TX.** Per §2.5, the FCS is computed before `tx_go`, between bursts. So the saving leaves the per-bit streaming budget alone (§8's n=2 zero-headroom analysis is unchanged) and shortens the dead time before each

frame instead. At n=2 / 250 MHz, a BERT burst is about 138 symbols × 10 bits × 2 cycles ≈ 2 760 core cycles on the wire:
    - The software CRC (≈9.5k cycles, 38 µs) took about 3.4× longer than the frame it protected.
    - The accelerator (181 cycles, 0.72 µs) takes about 7% of it.
    - The PRNG fill (≈1.5k cycles for 128 octets) is now the largest per-frame software cost.
- **RX.** `rx_crc_check` runs post-frame, before PRU1 re-arms. The ≈9.4k cycles saved come straight off the minimum inter-frame gap RX can accept.

These are simulator figures, not silicon measurements. The CRC model counts each `XOUT`/`XIN` as one cycle, and the `NOP` count follows the TRM's "1–2 NOPs" rule.

9.4 Verification

- `tests/test_pif_eth.py` adds three tests:
    - `test_firmware_uses_shared_hw_crc32_include`: all four firmware images include the hardware routine and none inlines a CRC.
    - `test_crc32_core_matches_zlib`: both the hardware and software `crc32_core` are checked against `zlib` for lengths 0–7, 60, 128 and 201, including that `r28`/`r29` survive the call.
    - `test_hw_crc32_core_is_much_cheaper_than_software`: a cycle-cost check.
- Full repo suite: 1407 passed, 2 xfailed.
- `python3 source/pif_eth/rx_driver.py o1` still passes all 28 rung/seed combinations (n_tx = 2, 4, 6, 8 × 7 seeds) with `ovf = symerr = biterr = 0` and `crc_ok = 1`, including the 125 Mbaud rung.

9.5 Silicon check: accumulator bit order (2026-10-07)

`crc_example.asm` was run standalone on AM243x silicon, built with `clpru`. It read `R10 = 0x0000C9D7` (CRC-16) and `R11 = 0x383AD48D` (CRC-32). The simulator returned `0x0000EB93` and `0xB12B5C1C`, the exact bit-reverse of each value.

- **Root cause.** The RTL bit-reverses each data word and feeds it into an MSB-first shift register with the normal polynomials (`references/icss_m_xfr_crc_functional_spec.md` §4). So data is consumed LSB first, but the accumulator, and with it the `CRC_DATA` (`R29`) read, is not reflected. The model had stored it reflected. The TRM agrees with silicon: for CRC-16 only `CRC_DATA[15:0]` and `CRC_DATA_32_BFLIP[31:16]` are valid, which only works for a non-reflected accumulator.
- **What reads what.** `R28` (`CRC_DATA_32_BFLIP`) is the reflected CRC that Ethernet and `zlib.crc32` use, before the final XOR. `R29` is the raw accumulator. `CRC_SEED` loads in the same raw order as `R29`.
- **Model fix.** `xfr/crc_accelerator.py` now keeps an MSB-first register (polynomials `0x8005`, `0x1021`, `0x04C11DB7`), fed LSB first per byte. Results do not depend on the write width.
- **Firmware fix.** `crc32_core` now takes the FCS from `R28`. It still reads `R29` before a byte-wide tail session, because that raw value is what `CRC_SEED` needs. The instructions are only reordered, so every cycle count in §9.3 is unchanged.
- **Examples.** `crc_example.asm` now expects `0x0000C9D7` / `0x383AD48D` (silicon values). `crc_bitswap_example.asm` now expects `R27 = 0x1C5C2BB1`, `R28 = 0xB12B5C1C`, `R29 = 0x383AD48D`.
- **Verification.** Full repo suite: 1436 passed, 2 xfailed. `rx_driver.py o1` passes all 28 rung/seed combinations. The `pif_eth` firmware itself (test plan TC4–TC7) has not been run on silicon yet.

The bug was invisible in the simulator: every test compared the model with a reference written under the same reflected assumption. The silicon run caught it.